



IETF-126 Hackathon

I2ICF Side Meeting



Interface to In-Network Computing Functions (I2ICF) Project

July 21, 2026, Vienna

Champion: Jaehoon (Paul) Jeong

Members: Mose Gu, Yoseop Ahn, Jisuk Chae, [EunJin Hwang](#), Kihyeon Shim, KyeongHoon Jeong, Joonsang Park, YoonJun Choi, Jisol Min

Department of Computer Science and Engineering at SKKU

Email: {pauljeong, ahnjs124, rna0415, sjw6136, sue030124, wangxudong28, eunijhwang}@skku.edu

IETF-126 Interface to In-Network Computing Functions (I2ICF)



IETF-126 Interface to In-Network Computing Functions (I2ICF) Project Professors:

- Jaehoon Paul Jeong (SKKU)
- Yong-Geun Hong (DJU)
- Joo-Sang Youn (DEU)

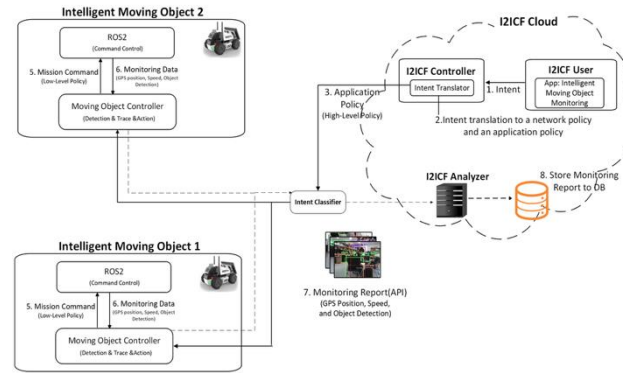
Researchers:

- Kehan Yao (China Mobile)
- Byoungman Robert An (KETI)
- Jung-Soo Park (ETRI)
- Yunchul Choi (ETRI)

Students:

- Mose Gu (SKKU)
- Yoseop Ahn (SKKU)
- Jisuk Chae (SKKU)
- Kihyun Shim (SKKU)
- Eunjin Hwang (SKKU)
- Keonghun Jeong (SKKU)
- Yoonjun Choi (SKKU)
- Joonsang Park (SKKU)
- Jisol Min (SKKU)

Champion: Jaehoon Paul Jeong (SKKU) I2ICF Framework



Objectives

- To demonstrate an intent-driven, closed-loop safety pipeline in which a moving object receives an intent, this I2ICF framework (i) performs on-board camera-based object detection and camera-LiDAR calibration/fusion for distance estimation, (ii) streams detections to a cloud server, and (iii) executes real-time stop/avoidance for obstacles and pedestrians.

Future Work

- To optimize End-to-End performance (e.g., inference accuracy, and inference latency) and extend to two robot cars that exchange on-board inference data to cooperatively plan collision-free, optimal paths when approaching each other.

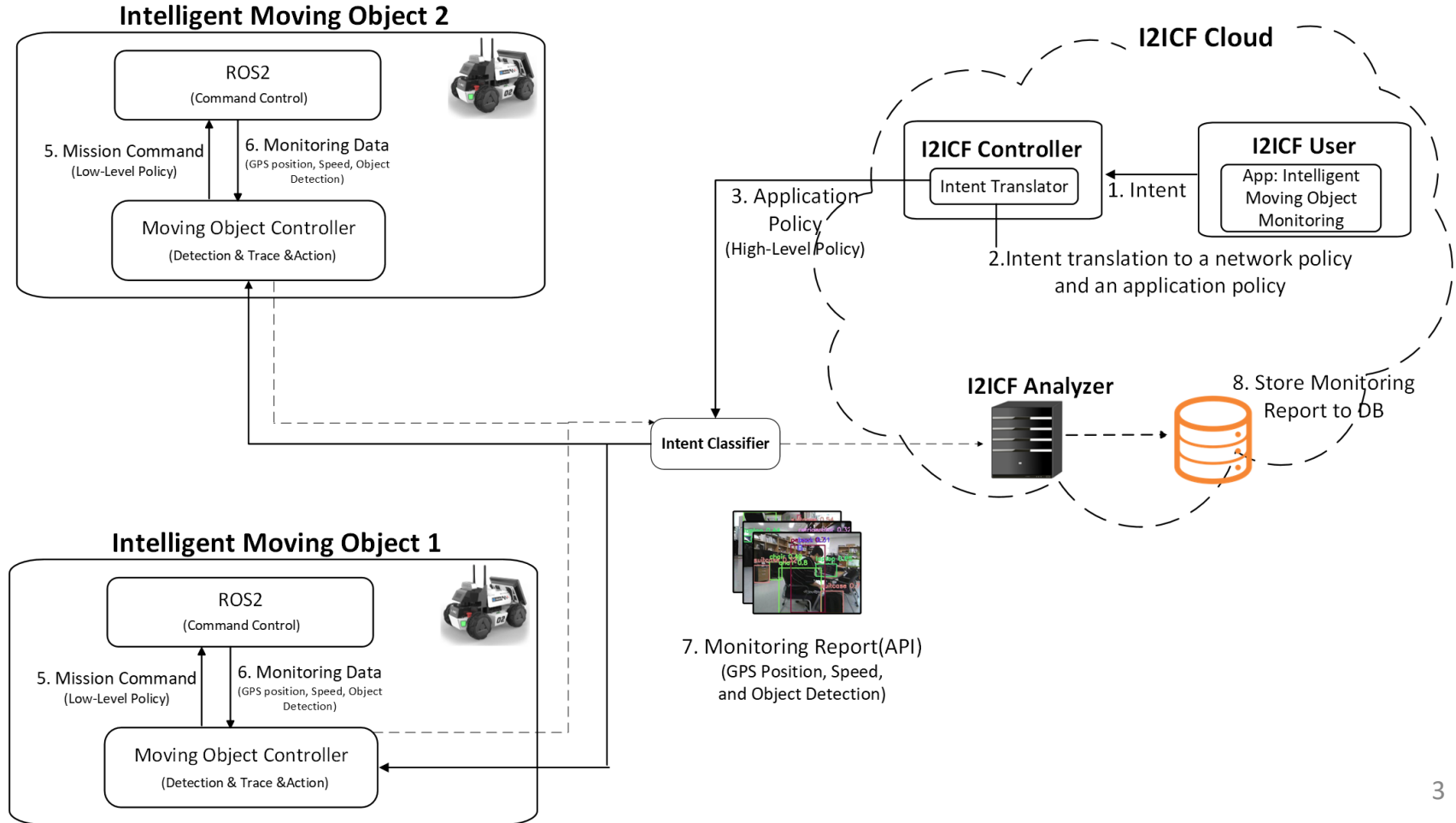
I2ICF Developing Environment

- OS: Ubuntu 20.04
- Kubernetes: Microk8s v1.32.2
- Object Detection: YOLO 11n
- LLM: Ollama (Llama 3.1 8B)
- ROS2 Version: Humble
- Backend: Django REST
- GitHub Repository:
<https://github.com/jaehoonpauljeong/I2ICF/tree/main/IETF-126>

Workflow of the I2ICF Testbed

- Intent Submission: A User submits a Trace/Detection/Action intent via web UI in natural language (e.g., "move forward and detour").
- Intent Translation with LLM: The I2ICF Controller passes the intent to Ollama (Llama 3.1 8B), which outputs a structured command of {action, speed, duration} as a high-level policy.
- Intent Classification: AgileX LIMO receives the translated intent and selects an appropriate action from the list.
- Action: LIMO executes the selected action via rosbridge WebSocket (/cmd_vel) if a new obstacle is detected.
- Perception & Detour: LIMO detects an object at 10 Hz using a camera with YOLO 11n, selects a detour path to avoid the obstacle, and continues to move.

Interface to In-Network Computing Functions (I2ICF) for Mobile Objects

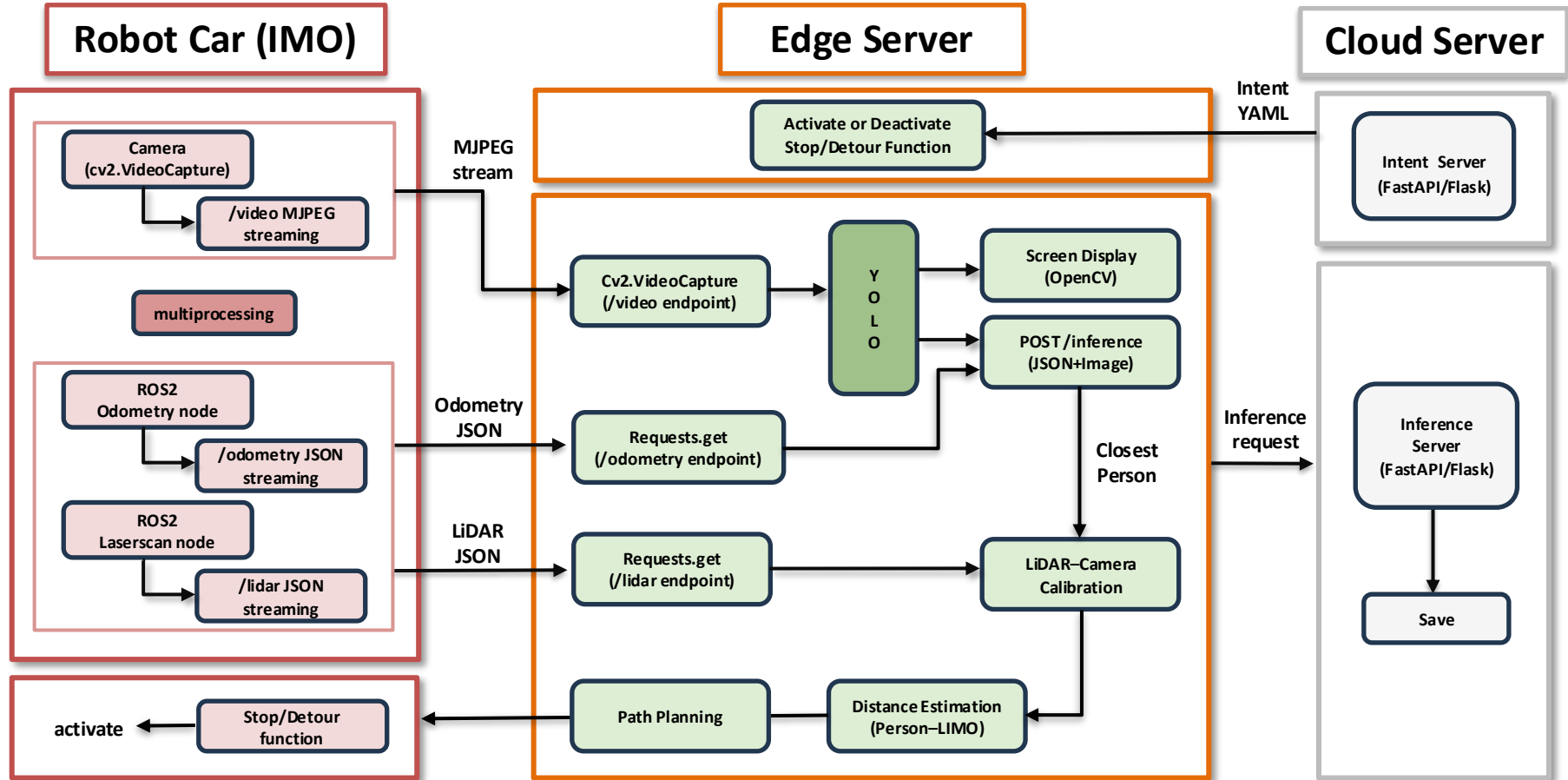


Goal of I2ICF Project

- The goal is to show the feasibility of the I2ICF framework that translates natural language intent into complex actions for a moving object
 - **I2ICF (Interface to In-Network-Computing Functions) Moving Object Controller**
 - An intent translation server converts complex user intents into a machine-readable policy document executable by the LIMO moving object and delivers it to the robot.
 - Upon receiving the translated policy document, the LIMO executes the specified actions and provides runtime status for execution monitoring
- Internet Drafts for the I2ICF Project
 - <https://datatracker.ietf.org/doc/draft-jeong-nmrg-ibn-network-management-automation/07/>
 - <https://datatracker.ietf.org/doc/draft-ahn-nmrg-5g-security-i2nsf-framework/02/>
 - <https://datatracker.ietf.org/doc/draft-gu-nmrg-intent-translator/03/>

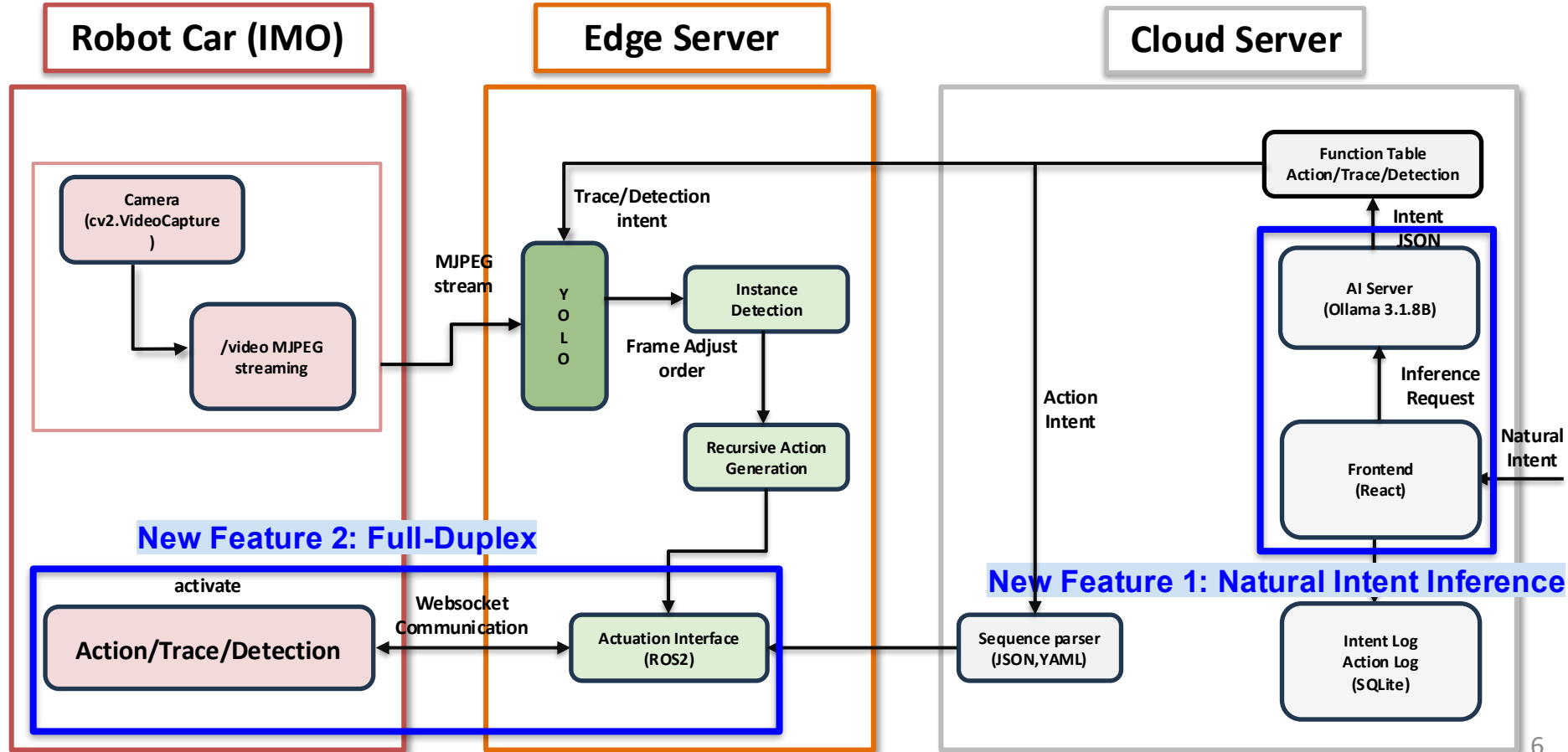
IETF-125 I2ICF Framework (Before)

: To show feasibility of self-determined collision avoidance for moving objects.



IETF-126 I2ICF Framework (Now)

: To translate natural language intent into complex actions



Demonstration (1/4)

```
agilex@agikex-NUC12WSK17: $ ros2 launch limo_base limo_base.launch.py
[INFO] [launch]: All log files can be found below /home/agilex/.ros/log/2026-07-13-13-29-05-606682-agikex-NUC12WSK17-3669
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [limo_base-1]: process started with pid [3670]
[limo_base-1] Loading parameters:
[limo_base-1] - port name: ttyUSB1
[limo_base-1] - odom frame name: odom
[limo_base-1] - base frame name: base_link
[limo_base-1] - pub odom tf: 0
[limo_base-1] - use_mcnamu: 0
[limo_base-1] [INFO] [1783920545.720714401] [limo_base_node]: connet the serial port: '/dev/ttyUSB1'
[limo_base-1] [INFO] [1783920545.723553852] [limo_base_node]: enableCommandedMode:
[limo_base-1] [INFO] [1783920545.723563347] [limo_base_node]: Open the serial port: '/dev/ttyUSB1'
```

1. [LIMO] Run 'limo_base.launch.py'

(Initializes the LIMO hardware interface and enables ROS 2-based motion control)

```
agilex@agikex-NUC12WSK17: ~
agilex@agikex-NUC12WSK17:~$ python3 ~/camera_stream.py
[Camera] 스트림 시작: http://192.168.50.165:8080
[Camera] 스냅샷: http://192.168.50.165:8080/snapshot
```

2. [LIMO] Run 'Camera stream.py'

(Captures camera frames and provides MJPEG streaming and snapshot endpoints)

```
Terminal
agilex@agikex-NUC12WSK17: ~
agilex@agikex-NUC12WSK17: $ ros2 launch rosbridge_server rosbridge_websocket.launch.xml
[INFO] [launch]: All log files can be found below /home/agilex/.ros/log/2026-07-13-13-31-37-011793-agikex-NUC12WSK17-4160
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [rosbridge_websocket-1]: process started with pid [4161]
[INFO] [rosapi_node-2]: process started with pid [4163]
[rosbridge_websocket-1] [WARN] [1783920697.451486700] [rosbridge_websocket]: The 'default_call_service_timeout' parameter is currently set to 0.0, which means service calls will block indefinitely if no response is received. Please note that in the Jazzy and later releases, the default value for this parameter will be updated to 5.0 seconds.
[rosbridge_websocket-1] [WARN] [1783920697.451696711] [rosbridge_websocket]: The 'call_services_in_new_thread' parameter is currently set to False, which means service calls will block the main thread. Please note that in the Jazzy and later releases, the default value for this parameter will be updated to True.
[rosbridge_websocket-1] [WARN] [1783920697.451845108] [rosbridge_websocket]: The 'send_action_goals_in_new_thread' parameter is currently set to False, which means sending action goals will block the main thread. Please note that in the Jazzy and later releases, the default value for this parameter will be updated to True.
[rosbridge_websocket-1] [INFO] [1783920697.453177019] [rosbridge_websocket]: Rosbridge WebSocket server started on port 9090
```

3. [LIMO] Run 'websocket'

(Enables real-time communication between the backend server and ROS 2 topics)

Demonstration (2/4)

Intent-Translator

Create-Intent



1. Natural-Language Intent Input



4. LIMO Command Execution

```
def natural_to_limo_sequence(text: str) -> list:
    prompt = f"""You are a controller for Agilex LIMO robot.
    The user gives a natural language command that may contain multiple sequential actions
    Parse it into an ordered list of steps and return JSON.

    Available commands:
    - "go straight" : move forward
    - "turn left" : rotate left
    - "turn right" : rotate right
    - "move back" : move backward
    - "stop" : stop all movement

    Rules:
    - Each command must be exactly one of the 5 options above
    - speed: 0.1 to 1.0 (default 0.5)
    - duration in seconds. "one block"=3.0s, "two blocks"=6.0s, "three blocks"=9.0s
    - Return a JSON object with a "steps" key containing the array

    Examples:
    Input: "go forward two blocks and turn left"
    Output: [{"steps": [{"command": "go straight", "speed": 0.5, "duration": 6.0}], [{"co

    Input: "앞으로 두 블록 가고 오른쪽으로 돌아"
    Output: [{"steps": [{"command": "go straight", "speed": 0.5, "duration": 6.0}], [{"co
```

2. LLM-Based Intent-to-JSON Conversion

```
class LimoDirectView(APIView):
    def post(self, request):
        text = request.data.get("text", "").strip()
        if not text:
            return Response({"error": "text field required"}, status=status.HTTP_400_B

        sequence = natural_to_limo_sequence(text)
        step_results = send_sequence_to_limo(sequence)
        all_success = all(r["success"] for r in step_results)

        results = []
        for step, result in zip(sequence, step_results):
            results.append({
                "command": step["command"],
                "speed": step["speed"],
                "duration": step["duration"],
                "cmd_vel": action_to_cmd_vel(step["command"]),
                "success": result["success"],
            })

        from django.utils import timezone
        safe_create(NaturalIntent, user="limo_direct", intent=text[:50], timestamp=ti

        return Response({
```

3. Robot Command Dispatch to LIMO

Demonstration (3/4)

Supported Features

```
29
30 class TraceSession:
31     def __init__(self):
32         self._running = False
33         self._target_class = "person"
34         self._thread: Optional[threading.Thread] = None
35
36     @property
37     def running(self) -> bool:
38         return self._running
39
40     def start(self, target_class: str = "person") -> None:
41         if self._running:
42             self.stop()
43         self._target_class = target_class
44         self._running = True
45         self._thread = threading.Thread(target=self._loop, daemon=True)
46         self._thread.start()
47         print(f"[Trace] Session started - target: {target_class}")
48
49     def stop(self) -> None:
50         self._running = False
51         if self._thread:
52             self._thread.join(timeout=3)
53         print("[Trace] Session stopped")
54
```

1.trace.py
(Object tracing)

```
31 _SEARCH_STEP = 0.1 # seconds per search tick
32
33
34 class DetectionSession:
35     def __init__(self):
36         self._running = False
37         self._thread: Optional[threading.Thread] = None
38
39     @property
40     def running(self) -> bool:
41         return self._running
42
43     def start(self) -> None:
44         if self._running:
45             return
46         self._running = True
47         self._thread = threading.Thread(target=self._loop, daemon=True)
48         self._thread.start()
49         print("[Detection] Session started")
50
51     def stop(self) -> None:
52         self._running = False
53         if self._thread:
54             self._thread.join(timeout=3)
55         print("[Detection] Session stopped")
56
57 # — internal —
```

2. detection.py
(Detect person and react)

Demonstration (4/4)

```
(venv) PS C:\Users\rrooy\RIHO-PROJECT\RIHO-PROJECT\backend> python manage.py runserver
[14/Jul/2026 11:55:48] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:55:51] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:55:54] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:55:57] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:55:58] "OPTIONS /api/infer/ HTTP/1.1" 200 0
[Infer] text='move forward and turn left' → intent={'mode': 'action', 'command': 'go straight', 'speed': 0.5, 'duration': 2.0, 'target_class': None}
[14/Jul/2026 11:56:00] "GET /api/status/ HTTP/1.1" 200 76
[SeqParser] go straight done (lx=0.5, az=0.0, dur=2.0s)
[Infer] step={'command': 'go straight', 'linear_x': 0.5, 'angular_z': 0.0, 'duration': 2.0, 'twist': {'linear': {'x': 0.5, 'y': 0.0, 'z': 0.0}, 'angular': {'x': 0.0, 'y': 0.0, 'z': 0.0}}} results=[{'command': 'go straight', 'success': True}]
[14/Jul/2026 11:56:02] "POST /api/infer/ HTTP/1.1" 200 365
[14/Jul/2026 11:56:03] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:06] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:09] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:12] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:15] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:18] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:21] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:25] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:28] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:31] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:34] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:37] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:40] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:43] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:45] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:49] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:52] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:55] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:56:58] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:57:01] "GET /api/status/ HTTP/1.1" 200 76
[14/Jul/2026 11:57:04] "GET /api/status/ HTTP/1.1" 200 76
```

1. Backend Processing Logs
(Shows the interpreted intent, generated robot command, and execution result)

```
agilex@agikex-NUC12W...
---
linear:
  x: 0.5
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 0.5
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---
linear:
  x: 0.0
  y: 0.0
  z: 0.0
angular:
  x: 0.0
  y: 0.0
  z: 0.0
---

and later releases, the default value for this parameter will be updated to 5.0 seconds.
[rosbridge_websocket-1] [WARN] [1783997599.804766234] [rosbridge_websocket]: The 'call_services_in_new_thread' parameter is currently set to False, which means service calls will block the main thread. Please note that in the Jazzy and later releases, the default value for this parameter will be updated to True.
[rosbridge_websocket-1] [WARN] [1783997599.804936524] [rosbridge_websocket]: The 'send_action_goals_in_new_thread' parameter is currently set to False, which means sending action goals will block the main thread. Please note that in the Jazzy and later releases, the default value for this parameter will be updated to True.
[rosbridge_websocket-1] [INFO] [1783997599.806919108] [rosbridge_websocket]: Rosbridge WebSocket server started on port 9090
[rosbridge_websocket-1] [INFO] [1783997601.730271228] [rosbridge_websocket]: Calling services in existing thread
[rosbridge_websocket-1] [INFO] [1783997601.730522862] [rosbridge_websocket]: Sending action goals in existing thread
[rosbridge_websocket-1] [INFO] [1783997601.731210787] [rosbridge_websocket]: Client connected. 1 clients total.
```

2. ROS 2 Command Reception via WebSocket
(Confirms that velocity commands are transmitted through WebSocket and received by the ROS 2 system)

What we learned

- **Enhanced LIMO's actuation based on simplified intent (e.g., Stop/Detour)**
 - We updated LIMO's intent understanding to include not only 'go', 'stop', and 'detour', but also more complex intent adaptation, such as detecting and chasing while detouring obstacles.
- **Natural Intent Translation to Policy Inference**
 - We utilized a Large Language Model (Llama 3.1) to extract key semantic representations from natural language inputs, subsequently generating a control policy to retrieve core actions from the predefined function table."
- **Integration of Full-Duplex Control Architecture**
 - We leveraged previous hackathon achievements_object detection and LiDAR distance calibration modules used for obstacle detouring and successfully integrated them into LIMO's intent-driven management system.

Next Step

- **Enhance Robustness**
 - Implement a fully automated, intent-driven management framework tailored for dynamic, moving objects.
- **Deploy an Agentic orchestrator**
 - Introduce a higher-level LLM as an agentic orchestrator to seamlessly orchestrate the entire system pipeline.
- **Optimize for Latency**
 - Execute speculative actions based on fast-path predictions while the brain AI reasons in parallel, followed by immediate command correction if any discrepancy arises.
 - Stream continuous camera frames via MJPEG from a LIMO robot car to an edge server to minimize latency for real-time YOLO perception.

Open-Source Project for IETF-126

<https://github.com/jaehoonpauljeong/I2ICF/tree/main/IETF-126/>

The screenshot shows the GitHub interface for the repository `jaehoonpauljeong / I2ICF`. The left sidebar displays the file tree with the `main` branch selected. The `IETF-126` directory is expanded, showing subdirectories like `intentString`, `intent_logs`, `intranslator`, `map`, and files like `.gitignore`, `manage.py`, and `mapping_rules.yaml`.

The main content area shows the commit history for the `IETF-126 / backend` directory. The commit message is "Add IETF-126 LIMO project: natural language to robot control via LLM ...". The commit history table is as follows:

Name	Last commit message	Last commit date
..		
intentString	Add IETF-126 LIMO project: natural language to robot control via LLM ...	5 hours ago
intent_logs	Add IETF-126 LIMO project: natural language to robot control via LLM ...	5 hours ago
intranslator	Add IETF-126 LIMO project: natural language to robot control via LLM ...	5 hours ago
map	Add IETF-126 LIMO project: natural language to robot control via LLM ...	5 hours ago
.gitignore	Add IETF-126 LIMO project: natural language to robot control via LLM ...	5 hours ago
manage.py	Add IETF-126 LIMO project: natural language to robot control via LLM ...	5 hours ago
mapping_rules.yaml	Add IETF-126 LIMO project: natural language to robot control via LLM ...	5 hours ago

I2ICF Team

- **Professors:**

- Jaehoon Paul Jeong (SKKU)
- Yong-Geun Hong (DJU)
- Joo-Sang Youn (DEU)

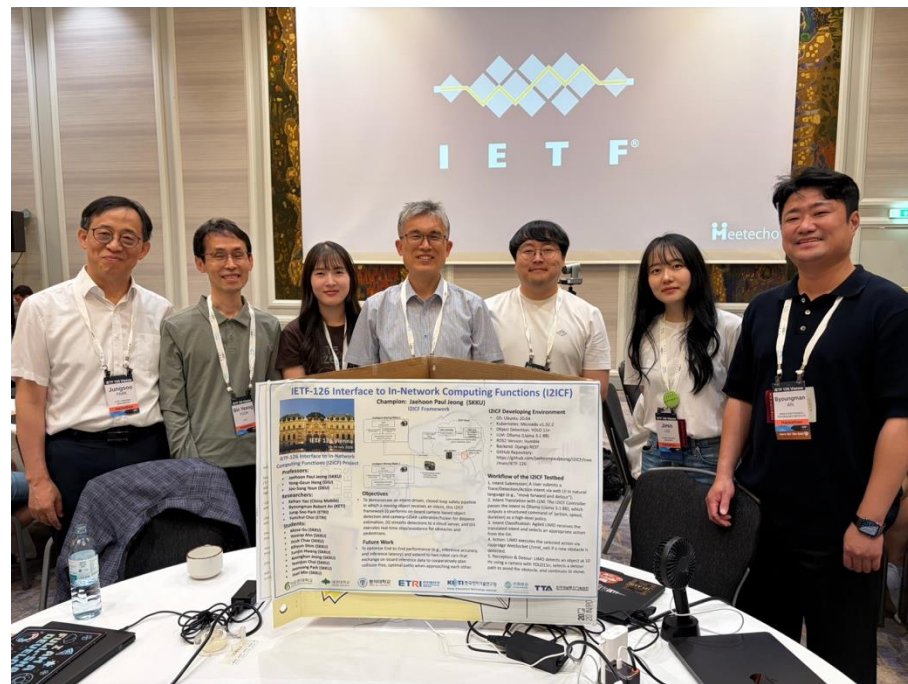
- **Researchers:**

- Kehan Yao (China Mobile)
- Byoungman Robert An (KETI)
- Jung-Soo Park (ETRI)
- Bin Young Yoon (ETRI)
- Yunchul Choi (ETRI)

- **Students:**

- Mose Gu (SKKU)
- Yoseop Ahn (SKKU)
- Jisuk Chae (SKKU)
- Kihyun Shim (SKKU)
- Eunjin Hwang (SKKU)
- Xudong Wang (SKKU)
- Keonghun Jeong (SKKU)
- Yoonjun Choi (SKKU)
- Joonsang Park (SKKU)
- Jisol Min (SKKU)

Hackathon Team Photo



Thank you!